

VisionInspect AI

Manufacturing Defect Detection & Quality Inspection System

Milestone 1 — Project Report

Akshaya Reddy

July 2026

Table of Contents

- 1. Project Description..... 3
- 2. Tech Stack..... 3
- 3. Database Architecture & Persistence..... 4
- 4. JWT Authentication & Security..... 6
- 5. Role-Based Access Control (RBAC)..... 7
- 6. Image Upload Workflow..... 8
- 7. Live Dashboard & Analytical Reports..... 9
- 8. Project Directory Structure..... 11
- 9. Conclusion..... 12

1. Project Description

VisionInspect AI is an AI-powered manufacturing quality inspection platform designed to detect product defects from images, manage quality inspection workflows, and provide live production analytics. This system acts as a major final-year project, utilizing a modern decoupled stack to automate and log industrial quality assurance.

The current implementation delivers the full application backbone: a secure multi-role backend, a persistent relational database, an operator-facing web interface, and a live analytics dashboard. This backbone is designed to support the future integration of the AI defect-detection model (PyTorch / OpenCV) which will plug directly into the existing image upload and inspection pipeline.

2. Tech Stack

Layer	Technology
Backend	FastAPI (Python 3.10+)
Frontend	React.js (Vite, React Router, Context API)
Styling	Tailwind CSS v3 (Dark Mode & Glassmorphism)
Database	PostgreSQL
ORM & Connection	SQLAlchemy 2.0 (type-safe declarative mapping)
Security	JWT Authentication (python-jose, passlib[bcrypt])
Access Control	Role-Based Access Control (RBAC)
Future AI Hook	PyTorch & OpenCV (planned integration; placeholders active)

3. Database Architecture & Persistence

A PostgreSQL database was integrated with optimized connection pooling (pool_size, max_overflow, pool_pre_ping, pool_recycle) to keep the backend resilient under repeated inspection requests. SQLAlchemy 2.0 ORM schemas were designed inside the backend/models/ package:

- users — manages operator accounts and credential verification.
- uploaded_images — logs every image captured on the production line.
- inspections — stores AI prediction classification outputs, confidence scores, and status flags.
- reports — stores generated compliance PDF export paths (one-to-one with inspections).

Database integrity constraints — foreign keys, indexes, and cascade delete rules — were defined across all four tables to keep inspection records consistent.

```
user@akshaya-reddy:~$ sudo -u postgres psql
psql (17.4 (Ubuntu 17.4-1.pgdg22.04+2))
Type "help" for help.

postgres=# \c visioninspect
You are now connected to database "visioninspect" as user "postgres".
visioninspect=# \dt
          List of relations
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 public | inspections    | table | postgres
 public | reports        | table | postgres
 public | uploaded_images | table | postgres
 public | users          | table | postgres
(4 rows)
```

Figure 1: psql session confirming the four core tables (inspections, reports, uploaded_images, users) created inside the visioninspect database.

id	full_name	email	password_hash	role	created_at
1	Akshaya Reddy	admin@visioninspect.com	\$2b\$12\$185W6gVsJ06X54dHyxrGyeNu1Comfqcnv33KhKXDT2uph1Mxzg6eq	admin	2026-07-09 01:16:03.573722+05:30
2	Test User	test@gmail.com	\$2b\$12\$K0B35kq2wyYRiCfQUYDVvERCo0/vDF5zQgMWvE7hfFrDdMD0sP.y	quality_engineer	2026-07-09 16:09:45.810035+05:30
3	test	akkireddy@gmail.com	\$2b\$12\$6I095Yh9.oJdfCKEj0MBPux0tQV/Lflgl5VXhMaaIRB0.xt6Iq58C	factory_supervisor	2026-07-09 16:18:39.654651+05:30
4	dimp1	dimp1reddy@gmail.com	\$2b\$12\$9dKaUa2IFb8K6YtP7UEBYOnWdJXI41Yzj/OqzBn58LtZ8HKTakzmy	quality_engineer	2026-07-09 16:19:49.865041+05:30

(4 rows)

Figure 2: users table populated with registered operator accounts, showing bcrypt-hashed passwords and assigned roles (admin, quality_engineer, factory_supervisor).

id	filename	filepath	uploaded_by	upload_at
1	Screenshot from 2026-07-02 00-24-36.png	/home/user/visioninspect-at/uploads/3f905c39bbc84e3184015001d67d46cb.png	1	2026-07-09 12:12:48.834427+05:30
2	Screenshot from 2026-07-07 22-54-52.png	/home/user/visioninspect-at/uploads/c3f15b5595a54b3595d0b1e54a8240d4.png	1	2026-07-09 15:37:59.913509+05:30
3	rlsks.png	/home/user/visioninspect-at/uploads/8f69adb29d0d4b6fb8fde046535036e.png	1	2026-07-09 16:53:13.305689+05:30
4	Screenshot from 2026-07-10 12-11-40.png	/home/user/visioninspect-at/uploads/0c81ebeb634677a1c9ac50d3e55d12.png	4	2026-07-10 17:21:11.744573+05:30
5	Screenshot from 2026-07-10 12-11-40.png	/home/user/visioninspect-at/uploads/973698e0c8014bad9b537e356f068bd8.png	4	2026-07-10 18:35:54.07858+05:30

(5 rows)

Figure 3: uploaded_images table logging each uploaded file's stored filename, disk filepath, uploader ID, and upload timestamp.

4. JWT Authentication & Security

Secure password hashing was configured using the bcrypt algorithm via passlib, and stateless session tokens are issued and validated using python-jose. The following endpoints were implemented inside routes/auth.py:

- POST /auth/register — validates and registers operator credentials.
- POST /auth/login — verifies passwords and issues an Access Token (60 min) and Refresh Token (7 days); integrates with standard OAuth2 forms for direct authorization in the Swagger docs.
- POST /auth/refresh — resolves access token renewal.
- POST /auth/logout — stateless logout stub.

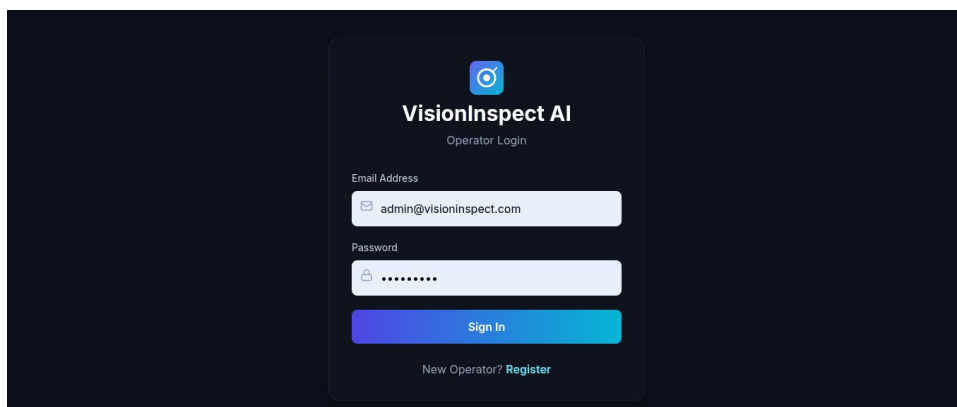


Figure 4: Operator Login screen — authenticates against the FastAPI /auth/login endpoint and stores the returned JWT.

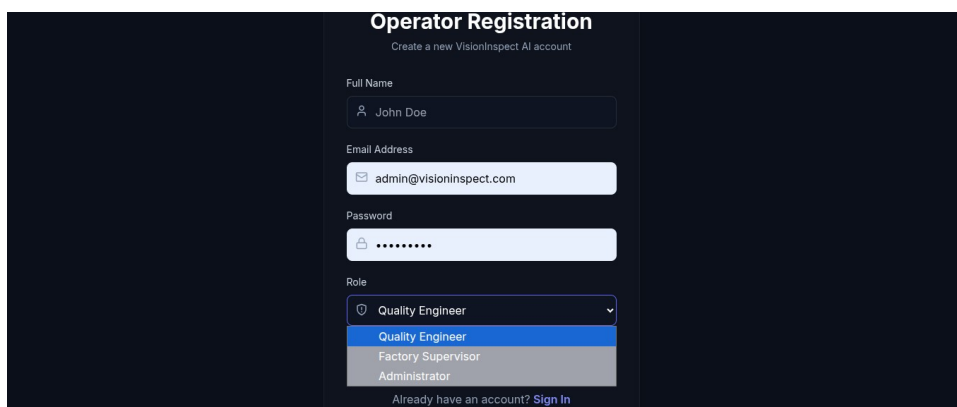


Figure 5: Operator Registration screen — new accounts are created with a selectable role (e.g. Quality Engineer) via /auth/register.

5. Role-Based Access Control (RBAC)

Strict roles and permission limits were established for the platform:

- Admin — create/delete users, manage roles, and view all reports.
- Quality Engineer (QE) — upload images, start inspections, and view own reports.
- Factory Supervisor (FS) — view dashboard metrics, view reports, and inspect production analytics.

Reusable dependency-injection guards enforce these boundaries server-side: RoleChecker asserts direct role membership, and PermissionChecker asserts role permissions against the permission map. Both raise structured HTTP 403 Forbidden responses when a request is unauthorized.

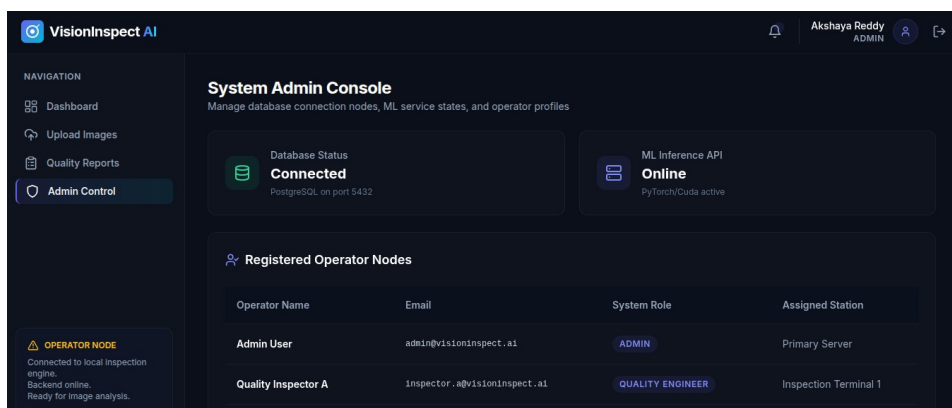


Figure 6: System Admin Console — lists registered operator nodes with their assigned system role and station, alongside live database and ML-inference service status.

6. Image Upload Workflow

Frontend Drag-and-Drop: a custom uploader (Upload.jsx) supports drag states, file-type checks (JPEG/PNG), file-size validation (max 5MB), and instant local previewing.

Backend API (POST /images/upload): validates incoming files, saves them under unique UUID filenames in the /uploads/ directory, commits metadata logs to PostgreSQL, and schedules a pending inspection slot — the hook that the future AI model will consume.

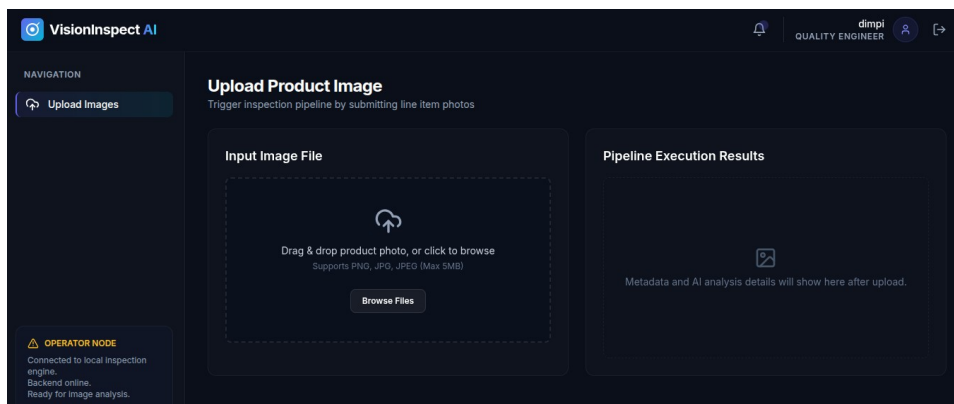


Figure 7: Upload Product Image screen — operators drag and drop a line-item photo to trigger the inspection pipeline.

7. Live Dashboard & Analytical Reports

Backend Analytics (GET /dashboard): runs database aggregation queries to calculate live metrics — Pass Rate, Defects Count, and Inspected Total — and returns the most recent inspection logs.

Backend Reports List (GET /reports): exposes completed inspection logs joined with their source image details.

Frontend Telemetry Integration: the dashboard and reports pages were refactored to pull live data straight from the backend, with the dashboard table streaming actual image thumbnails from the server's static /uploads mount.

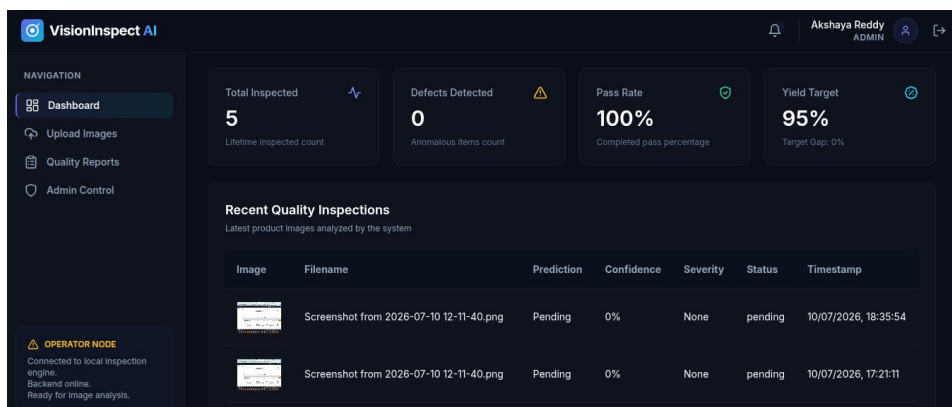


Figure 8: Operator dashboard showing total inspected count, defects detected, pass rate, yield target, and the recent quality inspections log.

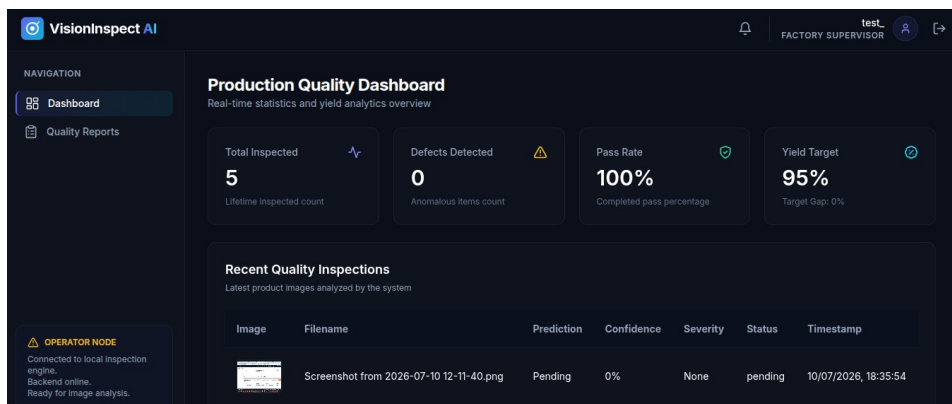


Figure 9: Production Quality Dashboard as seen from the Factory Supervisor role, displaying real-time yield analytics.

8. Project Directory Structure

The project follows a clean, decoupled backend/frontend layout so the two codebases can be developed, tested, and deployed independently:

```
visioninspect-ai/
├── .gitignore
├── README.md
├── PROJECT_OVERVIEW.md
├── backend/                                # Flat FastAPI backend namespace
│   ├── .env.example
│   ├── requirements.txt
│   ├── app.py                            # Main entry point (CORS, error filters, mounts)
│   ├── config.py                         # Pydantic Settings environment loader
│   ├── database.py                       # SQLAlchemy engine and session dependency
│   ├── dependencies.py                   # Security & role dependency guards
│   ├── security.py                       # JWT token and password hashing methods
│   ├── rbac.py                           # Role & Permission enums
│   ├── models/                           # SQLAlchemy 2.0 database tables
│   │   ├── user.py / image.py / inspection.py / report.py
│   ├── schemas/                           # Pydantic validation schemas
│   ├── routes/                           # auth.py, images.py, dashboard.py, reports.py
│   └── services/
│       └── user_service.py
├── frontend/                              # React.js frontend namespace (Vite)
│   ├── package.json / tailwind.config.js / vite.config.js
│   └── src/
│       ├── components/                   # Navbar, Sidebar, ProtectedRoute, RoleGuard
│       ├── context/AuthContext.jsx
│       ├── pages/                         # Login, Register, Dashboard, Upload, Reports, Admin
│       └── services/api.js               # Axios client with JWT header interceptors
```

9. Conclusion

The first milestone of VisionInspect AI successfully established the complete application backbone required before the AI defect-detection model is wired in. A PostgreSQL database with a properly constrained SQLAlchemy schema now persists users, uploaded images, inspections, and reports. JWT-based authentication and a three-tier RBAC system (Admin, Quality Engineer, Factory Supervisor) secure every endpoint, and a React frontend delivers a working login/registration flow, a drag-and-drop image upload workflow, an admin console, and a live analytics dashboard driven by real database aggregations rather than mock data.

This completed backbone provides a strong foundation for the next milestone, which will focus on integrating the PyTorch/OpenCV-based defect detection model into the existing upload → inspection pipeline, along with confidence scoring, anomaly heatmaps, and automated PDF compliance reporting.